



Module 1: Introduction to CMDB

1.1 What is CMDB?

- **CMDB (Configuration Management Database)** is a centralized database in ServiceNow that stores information about **Configuration Items (CIs)** and their **relationships**.
- CIs can be anything: servers, applications, network devices, users, business services, etc.

Example:

- A "Windows Server" CI might have attributes like:
 - Hostname = WIN-SVR-001
 - IP Address = 10.10.1.5
 - OS = Windows 2019
- It might be related to:
 - "HR Application" CI (Runs on this server)
 - "Database Server" CI (Depends on this server)

1.2 Why CMDB is Important?

- Provides a **single source of truth** for IT infrastructure.
- Helps ITSM processes:

- **Incident:** See which server an application runs on.
 - **Change:** Assess impact before making updates.
 - **Problem:** Find common failing CIs.
 - Helps ITOM processes (Discovery, Event Management, Monitoring).
-

1.3 Key CMDB Terminologies

1. **Configuration Item (CI)**
Any component that needs to be managed.
 - Example: A Laptop (`cmdb_ci_computer`), Router (`cmdb_ci_network`).
 2. **CI Class**
Defines a category/type of CI.
 - Example: `cmdb_ci_computer` is a subclass of `cmdb_ci`.
 3. **CI Attributes**
Properties of CIs.
 - Example: `serial_number`, `manufacturer`, `ip_address`.
 4. **CI Relationships**
Defines dependency.
 - Example: "Application A" **runs on** "Server B".
-

1.4 CMDB vs Asset Management

- **CMDB** → Focuses on IT services and their relationships.
- **Asset Management** → Focuses on financial, ownership, and lifecycle of assets.

Example:

- Laptop in Asset Mgmt → Purchase Date, Cost, Owner.
 - Same Laptop in CMDB → Installed Software, Network, Relationships.
-

1.5 ServiceNow CMDB Tables

- **Base Table:** `cmdb_ci` (all CIs extend this).
 - **Examples of Extended Tables:**
 - `cmdb_ci_computer` → Computers
 - `cmdb_ci_network` → Network Devices
 - `cmdb_ci_database` → Databases
-

1.6 Example: Creating a CI (Code & GUI)

Option 1: GUI

1. Navigate: **Configuration** → **Base Items** → **Computers**
 2. Click **New** → Fill fields:
 - Name = `WIN-SVR-001`
 - IP = `10.10.1.5`
 - OS = `Windows Server 2019`
 3. Save.
-

Option 2: Background Script

```
// Create a new Computer CI
var ci = new GlideRecord('cldb_ci_computer');
ci.initialize();
ci.name = "WIN-SVR-001";
ci.ip_address = "10.10.1.5";
ci.os = "Windows Server 2019";
ci.manufacturer = "Dell";
ci.insert();
```

Option 3: Script Include for CI Creation

```
var CMDBUtils = Class.create();
CMDBUtils.prototype = {
    initialize: function() {},

    createServerCI: function(name, ip, os) {
        var ci = new GlideRecord('cldb_ci_computer');
        ci.initialize();
        ci.name = name;
        ci.ip_address = ip;
        ci.os = os;
        return ci.insert();
    }
};

// Example call from background script
var utils = new CMDBUtils();
var sysId = utils.createServerCI("WIN-SVR-002", "10.10.1.10", "Windows
2022");
gs.print("New CI created with Sys ID: " + sysId);
```

1.7 Example: Creating a CI Relationship

Relationships are stored in `cldb_rel_ci`.

Background Script

```
// Create relationship: HR Application -> Runs on -> WIN-SVR-001
var rel = new GlideRecord('cmdb_rel_ci');
rel.initialize();
rel.parent = 'sys_id_of_HR_Application_CI'; // Parent CI
rel.child = 'sys_id_of_WIN_SVR_001';        // Child CI
rel.type = 'Runs on::Runs'; // Relationship type
rel.insert();
```

1.8 Real-Time Scenario

Scenario: HR application is down.

- Without CMDB → IT team checks 10+ servers manually.
 - With CMDB → From HR Application CI, open **Dependency View** and see it depends on **WIN-SVR-001**. Quickly check only that server.
-

Module 2: CMDB Data Model & Architecture

2.1 CMDB Architecture in ServiceNow

- CMDB is **table-based** → Every CI is a record in a CMDB table.
- **Inheritance model:**
 - **cmdb** (base class)
 - **cmdb_ci** (base for all CIs)

- `cmdb_ci_computer`
 - `cmdb_ci_network`
 - `cmdb_ci_database`
 - `cmdb_ci_app_server`
- Relationships stored in `cmdb_rel_ci`.
 - Visualization via **CMDB Schema Map** or **Dependency Views**.
-

2.2 CI Classes

- Each **class** represents a type of CI.
- Example hierarchy:
 - `cmdb_ci_computer` → extended by `cmdb_ci_server`
 - `cmdb_ci_network` → extended by `cmdb_ci_router`, `cmdb_ci_switch`
 - `cmdb_ci_storage_device`

GUI Step:

- Navigate: **Configuration** → **CI Class Manager**
 - Search for “Computer” → See hierarchy & attributes.
-

2.3 CI Attributes

- Properties that describe the CI.

- Example for `cmdb_ci_computer`:

- `name`
- `ip_address`
- `os`
- `manufacturer`
- `serial_number`

 GUI Step:

- Open `cmdb_ci_computer` → Configure → Form Layout → View attributes.
-

2.4 CI Relationships

- Relationships are critical for **impact analysis**.
- Stored in `cmdb_rel_ci` table.
- Relationship types are defined in `cmdb_rel_type`.

Examples:

- Application → **Runs on** → Server
- Server → **Depends on** → Database
- Switch → **Connected to** → Router

 GUI Step:

- Navigate: **Configuration** → **CI Relationship Types**

- Example: “Runs on::Runs”.
-

2.5 CMDB Schema Map

- Schema map provides a **visual representation** of CMDB classes & relationships.
- Example: Select `cmdb_ci_computer` → It shows parent (`cmdb_ci`) and children (`cmdb_ci_server`, `cmdb_ci_laptop`).

GUI Step:

- Navigate: **Configuration** → **CMDB Schema Map** → Search for `Computer`.
-

2.6 Identification & Reconciliation Engine (IRE)

- Prevents duplicate CIs.
- Uses **identification rules** to uniquely identify CIs.
- Uses **reconciliation rules** to control which data source updates which attributes.

GUI Step:

- Navigate: **Configuration** → **CI Identifier** → Example: `cmdb_ci_computer` identifier rule → matches by `serial_number` or `fqdn`.
-

2.7 Example: Querying CMDB with Code

Background Script: Fetch all Servers

```
var gr = new GlideRecord('cmdb_ci_server');
```



```
gr.query();

while(gr.next()){

    gs.print("Server: " + gr.name + " | IP: " + gr.ip_address);

}
```

Script Include: Utility to Fetch Relationships

```
var CMDBHelper = Class.create();

CMDBHelper.prototype = {

    initialize: function() {},

    getChildCIs: function(parentSysId) {

        var rel = new GlideRecord('cldb_rel_ci');

        rel.addQuery('parent', parentSysId);

        rel.query();

        var result = [];

        while(rel.next()) {

            result.push(rel.child.name.toString());

        }

        return result;

    }

};
```

```
// Example call

var helper = new CMDBHelper();

var apps = helper.getChildCIs('sys_id_of_server');

gs.print("Applications running: " + apps.join(', '));
```

GlideAggregate Example: Count by CI Type

```
var agg = new GlideAggregate('cmdb_ci');

agg.groupBy('sys_class_name');

agg.addAggregate('COUNT');

agg.query();

while(agg.next()) {

    gs.print(agg.sys_class_name + " → " + agg.getAggregate('COUNT'));

}
```

 Output Example:

```
cmdb_ci_computer → 120
```

```
cmdb_ci_network → 45
```

```
cmdb_ci_database → 10
```

2.8 Real-Time Scenario

Scenario:

A company wants to know:

- Which applications are running on which servers.
- If a server goes down, which business services are impacted.

Solution using CMDB Data Model:

1. CI Class → `cmdb_ci_app_server` for applications.
2. Relationship → “Runs on” → Server (`cmdb_ci_server`).
3. Query Builder → Drag Server + Application + Business Service → build dependency map.

 GUI Step:

- Navigate: **Configuration** → **Query Builder** → Create query to show *Application* → *Server* → *Business Service*.
-

2.9 Best Practices for CMDB Data Model

- Always use **CI Class Manager** for extending classes.
 - Use **identification rules** to avoid duplicates.
 - Do not overload CIs with non-essential attributes.
 - Maintain **relationships** – they are critical for impact analysis.
-

 End of Module 2 Summary

- Understood CMDB architecture & inheritance.
- Learned about CI classes, attributes, and relationships.

- Hands-on: Create & query CIs, build relationships (GUI + Code).
- Explored Schema Map, IRE, and Query Builder.